

# DEBERTA: 具有解缠注意力机制的解码增强型 BERT

Pengcheng He<sup>1</sup>, Xiaodong Liu<sup>2</sup>, Jianfeng Gao<sup>2</sup>, Weizhu Chen<sup>1</sup>

<sup>1</sup> 微软 Dynamics 365 AI <sup>2</sup> 微软研究院

{潘贺, 肖德龙, 高俊峰, 陈伟忠}@microsoft.com

## 摘要

近期预训练神经语言模型的进展显著提升了众多自然语言处理（NLP）任务的表现。在本文中，我们提出了一种新的模型架构 DeBERTa（解码增强型带解耦注意力机制的 BERT），通过两种新颖的技术改进了 BERT 和 RoBERTa 模型。第一种是解耦注意力机制，其中每个词都用两个向量分别表示其内容和位置，词与词之间的注意力权重分别通过基于内容和相对位置的解耦矩阵计算得出。第二种是在解码层使用增强型掩码解码器，将绝对位置信息纳入其中，以预测模型预训练中的掩码标记。此外，还采用了一种新的虚拟对抗训练方法进行微调，以提高模型的泛化能力。我们表明，这些技术显著提高了模型预训练的效率以及自然语言理解（NLU）和自然语言生成（NLG）下游任务的表现。与 RoBERTa-Large 相比，仅使用一半训练数据训练的 DeBERTa 模型在众多自然语言处理任务中表现始终更优，在 MNLI 上提升了 0.9%（从 90.2% 提升至 91.1%），在 SQuAD v2.0 上提升了 2.3%（从 88.4% 提升至 90.7%），在 RACE 上提升了 3.6%（从 83.2% 提升至 86.8%）。值得注意的是，我们通过训练一个包含 48 个 Transformer 层、拥有 15 亿参数的更大版本的 DeBERTa 模型来对其进行扩展。显著的性能提升使得单个 DeBERTa 模型首次在 SuperGLUE 基准测试（Wang 等人，2019a）的宏平均得分上超越了人类表现（89.9 对比 89.8），而截至 2021 年 1 月 6 日，集成的 DeBERTa 模型在 SuperGLUE 排行榜上位居榜首，大幅超越了人类基线（90.3 对比 89.8）。预训练的 DeBERTa 模型和源代码已发布在：<https://github.com/microsoft/DeBERTa1>。

## 1 简介

Transformer 已成为神经语言建模中最有效的神经网络架构。与按顺序处理文本的循环神经网络（RNN）不同，Transformer 采用自注意力机制并行计算输入文本中每个单词的注意力权重，该权重衡量每个单词对其他单词的影响，因此在大规模模型训练方面比 RNN 更具并行性（Vaswani 等人，2017）。自 2018 年以来，我们见证了大量基于 Transformer 的预训练语言模型（PLM）的兴起，例如 GPT（Radford 等人，2019；Brown 等人，2020）、BERT（Devlin 等人，2019）、RoBERTa（Liu 等人，2019c）、XLNet（Yang 等人，2019）、UniLM（Dong 等人，2019）、ELECTRA（Clark 等人，2020）、T5（Raffel 等人，2020）、ALUM（Liu 等人，2020）、StructBERT（Wang 等人，2019c）和 ERINE（Sun 等人，2019）。这些 PLM 通过使用特定任务的标签进行微调，在许多下游自然语言处理（NLP）任务中创造了新的最佳水平（Liu 等人，2019b；Minaee 等人，2020；Jiang 等人，2020；He 等人，2019a；b；Shen 等人，2020）。

<sup>1</sup> Our code and models are also available at HuggingFace Transformers: <https://github.com/huggingface/transformers>, <https://huggingface.co/models?filter=deberta>

在本文中，我们提出了一种新的基于 Transformer 的神经语言模型 DeBERTa（解码增强型带解缠注意力机制的 BERT），该模型通过两种新颖的技术改进了现有的最先进的预训练语言模型：一种是解缠注意力机制，另一种是增强型掩码解码器。

解缠注意力。与 BERT 中每个输入层的单词都用其词（内容）嵌入和位置嵌入之和表示的向量来表示不同，在 DeBERTa 中，每个单词都用两个向量分别表示其内容和位置，并且单词之间的注意力权重是基于它们的内容和相对位置分别通过解耦矩阵计算得出的。这一设计的动机在于观察到单词对之间的注意力权重不仅取决于它们的内容，还取决于它们的相对位置。例如，“深度”和“学习”这两个词在相邻出现时的依赖关系要比它们出现在不同句子时强得多。

增强型掩码解码器。与 BERT 类似，DeBERTa 也是通过掩码语言模型（MLM）进行预训练的。MLM 是一种填空任务，模型需要根据掩码标记周围的单词来预测被掩码的单词应该是什么。DeBERTa 在掩码语言模型中使用了上下文单词的内容和位置信息。解缠注意力机制已经考虑了上下文单词的内容和相对位置，但没有考虑这些单词的绝对位置，而在很多情况下，绝对位置对于预测至关重要。以句子“一家新商店在新购物中心旁边开业”为例，其中“商店”和“购物中心”这两个词被加粗标记以供预测。尽管这两个词的局部上下文相似，但它们在句子中扮演着不同的句法角色。（例如，句子的主语是“商店”而不是“购物中心”。）这些句法上的细微差别在很大程度上取决于单词在句子中的绝对位置，因此在语言建模过程中考虑单词的绝对位置非常重要。DeBERTa 在 softmax 层之前引入了绝对词位置嵌入，在该层中，模型基于单词内容和位置的聚合上下文嵌入来解码被遮蔽的单词。

此外，我们提出了一种新的虚拟对抗训练方法，用于对预训练语言模型进行微调以应用于下游自然语言处理任务。该方法在提高模型泛化能力方面效果显著。

我们通过一项全面的实证研究证明，这些技术显著提高了预训练的效率以及下游任务的表现。在自然语言理解（NLU）任务中，与 RoBERTa-Large 相比，使用一半训练数据训练的 DeBERTa 模型在广泛的自然语言处理任务中表现始终更优，在 MNLI 上提高了 0.9%（从 90.2% 提升至 91.1%），在 SQuAD v2.0 上提高了 2.3%（从 88.4% 提升至 90.7%），在 RACE 上提高了 3.6%（从 83.2% 提升至 86.8%）。在自然语言生成（NLG）任务中，DeBERTa 在 Wikitext-103 数据集上的困惑度从 21.6 降低到了 19.5。我们进一步扩大了 DeBERTa 的规模，通过预训练一个包含 48 层 Transformer 结构、拥有 15 亿参数的更大模型。单个 15 亿参数的 DeBERTa 模型在 SuperGLUE 基准测试（Wang 等人，2019a）上比拥有 110 亿参数的 T5 模型高出 0.6%（从 89.9% 提升至 89.3%），并且首次超越了人类基线（从 89.9% 提升至 89.8%）。截至 2021 年 1 月 6 日，DeBERTa 集成模型在 SuperGLUE 领先榜上位居榜首，大幅超越了人类基线（从 89.8% 提升至 90.3%）。

## 2 背景

### 2.1 变压器

基于 Transformer 的语言模型由堆叠的 Transformer 块组成（Vaswani 等人，2017 年）。每个块包含一个多头自注意力层，后面跟着一个全连接的位置前馈网络。标准的自注意力机制缺乏一种自然的方式来编码单词的位置信息。因此，现有的方法会在每个输入词嵌入中添加位置偏差，这样每个输入词就由一个其值取决于内容和位置的向量来表示。位置偏差可以通过绝对位置嵌入（Vaswani 等人，2017 年；Radford 等人，2019 年；Devlin 等人，2019 年）或相对位置嵌入（Huang 等人，2018 年；Yang 等人，2019 年）来实现。研究表明，相对位置表示对于自然语言理解和生成任务更有效（Dai 等人，2019 年；Shaw 等人，2018 年）。所提出的解缠注意力机制与所有现有方法不同，我们使用两个独立的向量分别表示每个输入词的内容和位置，并且

单词之间的注意力权重是分别通过其内容和相对位置的解耦矩阵来计算的。

## 2.2 掩码语言模型

大规模基于 Transformer 的预训练语言模型 (PLMs) 通常在大量文本上进行预训练, 以使用一种称为掩码语言模型 (MLM) 的自监督目标来学习上下文中的词表示 (Devlin 等人, 2019)。具体来说, 给定一个序列  $X$  “tx<sub>i</sub>u”, 我们将其破坏为  $X$

通过随机遮蔽 15% 的标记, 然后训练一个由参数  $\theta$  定义的语言模型, 根据  $X$  来预测被遮蔽的标记  $x$  从而重构  $X$ :

$$\max_{\theta} \log p_{\theta}(X|\tilde{X}) = \max_{\theta} \sum_{i \in C} \log p_{\theta}(\tilde{x}_i = x_i | \tilde{X}) \quad (1)$$

其中  $C$  是序列中被掩码标记的索引集。BERT 的作者建议将 10% 的掩码标记保持不变, 另外 10% 替换为随机选取的标记, 其余的则替换为 [MASK] 标记。

## 3 DEBERTA 架构

### 3.1 分离注意力: 一种用于内容和位置嵌入的双矢量方法

对于序列中位置为  $i$  的标记, 我们使用两个向量  $\mathbf{t}_{\text{H}i}$  和  $\mathbf{t}_{\text{P}i}$  来表示它, 这两个向量分别表示其内容以及与位置为  $j$  的标记的相对位置。标记  $i$  和  $j$  之间的交叉注意力得分的计算可以分解为四个部分, 具体如下:

$$\begin{aligned} A_{i,j} &= \{\mathbf{H}_i, \mathbf{P}_{i|j}\} \times \{\mathbf{H}_j, \mathbf{P}_{j|i}\}^{\top} \\ &= \mathbf{H}_i \mathbf{H}_j^{\top} + \mathbf{H}_i \mathbf{P}_{j|i}^{\top} + \mathbf{P}_{i|j} \mathbf{H}_j^{\top} + \mathbf{P}_{i|j} \mathbf{P}_{j|i}^{\top} \end{aligned} \quad (2)$$

也就是说, 一对词的注意力权重可以通过使用内容和位置的解耦矩阵计算四个注意力得分的总和来得出, 这四个注意力得分分别是内容到内容、内容到位置、位置到内容以及位置到位置的注意力得分。

现有的相对位置编码方法使用单独的嵌入矩阵来计算注意力权重中的相对位置偏差 (Shaw 等人, 2018 年; Huang 等人, 2018 年)。这相当于仅使用方程 2 中的内容到内容和内容到位置项来计算注意力权重。我们认为位置到内容项也很重要, 因为一对词的注意力权重不仅取决于它们的内容, 还取决于它们的相对位置, 而这一点只有同时使用内容到位置和位置到内容项才能充分建模。由于我们使用相对位置嵌入, 位置到位置项没有提供太多额外信息, 在我们的实现中从方程 2 中移除了。

以单头注意力为例, 标准的自注意力操作 (Vaswani 等人, 2017 年) 可以表述为:

$$\begin{aligned} Q &= H W_q, K = H W_k, V = H W_v, A = \frac{Q K^{\top}}{\sqrt{d}} \\ H_o &= \text{softmax}(A) V \end{aligned}$$

其中,  $H \in \mathbb{R}^{N \times d}$  表示输入的隐藏向量,  $H_o \in \mathbb{R}^{N \times d}$  表示自注意力的输出,  $W_q, W_k, W_v \in \mathbb{R}^{d \times d}$  为投影矩阵,  $A \in \mathbb{R}^{N \times N}$  为注意力矩阵,  $N$  为输入序列的长度,  $d$  为隐藏状态的维度。

记  $k$  为最大相对距离,  $\delta_{i,j} \in \mathbb{R}$  表示从标记  $i$  到标记  $j$  的相对距离, 其定义为:

$$\delta(i, j) = \begin{cases} 0 & \text{for } i - j \leq -k \\ 2k - 1 & \text{for } i - j \geq k \\ i - j + k & \text{others.} \end{cases} \quad (3)$$

<sup>2</sup>In this sense, our model shares some similarity to Tensor Product Representation (Smolensky, 1990; Schlag et al., 2019; Chen et al., 2019) where a word is represented using a tensor product of its filler (content) vector and its role (position) vector.

我们可以将带相对位置偏差的解缠自注意力表示为方程 4，其中  $Q_c$ 、 $K_c$  和  $V_c$  分别是使用投影矩阵  $W_{q,c}$ 、 $W_{k,c}$ 、 $W_{v,c}$  生成的投影内容向量， $P \in \mathbb{R}^{2k \times d}$  表示在所有层中共有的相对位置嵌入向量（即在前向传播过程中保持固定），而  $Q_r$  和  $K_r$  分别是使用投影矩阵  $W_{q,r}$ 、 $W_{k,r}$  生成的投影相对位置向量。

$$\begin{aligned}
 Q_c &= HW_{q,c}, K_c = HW_{k,c}, V_c = HW_{v,c}, Q_r = PW_{q,r}, K_r = PW_{k,r} \\
 \tilde{A}_{i,j} &= \underbrace{Q_i^c K_j^{c\top}}_{\text{(a) content-to-content}} + \underbrace{Q_i^c K_{\delta(i,j)}^r}_{\text{(b) content-to-position}} + \underbrace{K_j^c Q_{\delta(j,i)}^r}_{\text{(c) position-to-content}} \\
 H_o &= \text{softmax}\left(\frac{\tilde{A}}{\sqrt{3d}}\right)V_c
 \end{aligned} \tag{4}$$

$A_{ij}$  是注意力矩阵  $A$  的元素，代表从标记  $i$  到标记  $j$  的注意力得分。 $Q_i$  是  $Q_c$  的第  $i$  行。 $K_j$  是  $K_c$  的第  $j$  行。 $K_{\delta(i,j)}$  是  $K_r$  中的第  $\delta(i,j)$  行，其中

关于相对距离  $\delta(i,j)$ ， $Q_{\delta(i,j)}^r$  是  $Q_r$  中关于相对距离  $\delta(i,j)$  的第  $\delta(i,j)$  行。距离  $\delta(i,j)$ ，请注意，这里我们使用  $\delta(i,j)$  而不是  $\delta(j,i)$ 。这是因为对于给定的位置  $i$ ，位置到内容计算的是位置  $j$  处关键内容相对于查询位置  $i$  的注意力权重，因此相对距离为  $\delta(i,j)$ 。位置到内容项的计算方式为  $K_j^c Q_{\delta(j,i)}^r$ ，内容到位置项的计算方式与此类似。

最后，我们在  $A$  上应用一个  $\gamma_{3d}$  缩放因子。

□

该因素对于稳定模型很重要。

训练 (Vaswani 等人, 2017 年)，尤其是对于大规模的预训练语言模型。

---

#### Algorithm 1 Disentangled Attention

---

**Input:** Hidden state  $H$ , relative distance embedding  $P$ , relative distance matrix  $\delta$ . Content projection matrix  $W_{k,c}$ ,  $W_{q,c}$ ,  $W_{v,c}$ , position projection matrix  $W_{k,r}$ ,  $W_{q,r}$ .

```

1:  $K_c = HW_{k,c}$ ,  $Q_c = HW_{q,c}$ ,  $V_c = HW_{v,c}$ ,  $K_r = PW_{k,r}$ ,  $Q_r = PW_{q,r}$ 
2:  $A_{c \rightarrow c} = Q_c K_c^\top$ 
3: for  $i = 0, \dots, N - 1$  do
4:    $\tilde{A}_{c \rightarrow p}[i, :] = Q_c[i, :] K_r^\top$ 
5: end for
6: for  $i = 0, \dots, N - 1$  do
7:   for  $j = 0, \dots, N - 1$  do
8:      $A_{c \rightarrow p}[i, j] = \tilde{A}_{c \rightarrow p}[i, \delta[i, j]]$ 
9:   end for
10: end for
11: for  $j = 0, \dots, N - 1$  do
12:    $\tilde{A}_{p \rightarrow c}[:, j] = K_c[j, :] Q_r^\top$ 
13: end for
14: for  $j = 0, \dots, N - 1$  do
15:   for  $i = 0, \dots, N - 1$  do
16:      $A_{p \rightarrow c}[i, j] = \tilde{A}_{p \rightarrow c}[\delta[j, i], j]$ 
17:   end for
18: end for
19:  $\tilde{A} = A_{c \rightarrow c} + A_{c \rightarrow p} + A_{p \rightarrow c}$ 
20:  $H_o = \text{softmax}\left(\frac{\tilde{A}}{\sqrt{3d}}\right)V_c$ 

```

**Output:**  $H_o$

---

##### 3.1.1 高效实施

对于长度为  $N$  的输入序列，存储每个标记的相对位置嵌入需要的空间复杂度为  $O(N^2 d)$  (Shaw 等人, 2018 年; Huang 等人, 2018 年; Dai 等人, 2019 年)。然而，以内容到位置为例，我们注意到由于  $\delta(i,j) \in [0, 2k]$  以及嵌入

如果所有可能的相对位置始终是  $K_r P R^{2k \times d}$  的子集，那么我们就可以在所有查询的注意力计算中重复使用  $K_r$ 。

在我们的实验中，我们将预训练的最大相对距离  $k$  设为 512。利用算法 1 可高效计算出解耦的注意力权重。设  $\delta$  为根据公式 3 得出的相对位置矩阵，即  $\delta_{ri, js} = \delta_{pi, jq}$ 。我们不是为每个查询分配不同的相对位置嵌入矩阵，而是将每个查询向量  $Q_{ri}, :s$  乘以  $K_r P R^{d \times 2k}$ ，如第 3 至 5 行所示。然后，我们利用相对位置矩阵  $\delta$  作为索引提取注意力权重，如第 6 至 10 行所示。为了计算位置到内容的注意力得分，我们计算  $A_{pNc} r, js$ ，即，也就是通过将注意力矩阵  $A_{pNc}$  的每一列向量与相应的键向量  $K_{cj}, :s$  相乘得到  $Q_r$ ，如下所示  
第 11 行至第 13 行：最后，我们通过相对位置矩阵  $\delta$  提取相应的注意力得分作为索引，如第 14 行至第 18 行所示。这样，我们就不必为每个查询分配内存来存储相对位置嵌入，从而将空间复杂度降低到  $O_{pkdq}$ （用于存储  $K_r$  和  $Q_r$ ）。

### 3.2 增强型掩码解码器考虑了绝对词位置

DeBERTa 是通过掩码语言模型（MLM）进行预训练的，在这种训练方式中，模型被训练利用掩码标记周围的单词来预测被掩码的单词应该是什么。DeBERTa 在掩码语言模型中使用上下文单词的内容和位置信息。解耦注意力机制已经考虑了上下文单词的内容和相对位置，但没有考虑这些单词的绝对位置，而在很多情况下，绝对位置对于预测至关重要。

给定句子“一家新商店开在新购物中心旁边”，其中“商店”和“购物中心”被遮蔽以进行预测。仅依靠局部上下文（例如相对位置和周围单词）不足以让模型区分该句中的“商店”和“购物中心”，因为它们都位于“新”之后且相对位置相同。为了解决这一局限性，模型需要考虑绝对位置，将其作为相对位置的补充信息。例如，该句的主语是“商店”而非“购物中心”。这些句法上的细微差别在很大程度上取决于单词在句子中的绝对位置。

引入绝对位置有两种方法。BERT 模型在输入层引入绝对位置。在 DeBERTa 中，我们在所有 Transformer 层之后但在用于掩码标记预测的 *softmax* 层之前引入绝对位置，如图 2 所示。这样，DeBERTa 在所有 Transformer 层中捕获相对位置，并且仅在解码掩码词时将绝对位置作为补充信息使用。因此，我们将 DeBERTa 的解码组件称为增强型掩码解码器（EMD）。在实证研究中，我们比较了这两种引入绝对位置的方法，并观察到 EMD 效果更好。我们推测，BERT 早期引入绝对位置的方式可能不利于模型充分学习相对位置的信息。此外，EMD 还使我们能够在预训练时引入除位置之外的其他有用信息，这部分内容留待未来研究。

## 4 尺度不变微调

本节介绍了一种新的虚拟对抗训练算法——尺度不变微调（Scale-invariant-Fine-Tuning, SiFT），它是 Miyato 等人（2018 年）和 Jiang 等人（2020 年）所描述算法的一个变体，用于微调。

虚拟对抗训练是一种用于提升模型泛化能力的正则化方法。它通过增强模型对对抗样本的鲁棒性来实现这一目标，对抗样本是通过输入进行微小扰动而生成的。该方法对模型进行正则化处理，使得模型在处理特定任务的示例时，其输出分布与处理该示例的对抗扰动样本时的输出分布相同。

对于自然语言处理任务，扰动是施加在词嵌入上，而非原始的词序列上。然而，不同单词和模型的嵌入向量的值范围（范数）各不相同。对于拥有数十亿参数的大型模型，这种差异会更大，从而导致对抗训练的某些不稳定性。

受层归一化 (Ba 等人, 2016 年) 的启发, 我们提出了 SiFT 算法, 通过将扰动应用于归一化的词嵌入来提高训练的稳定性。具体来说, 在我们的实验中, 当对 DeBERTa 进行微调以适应下游自然语言处理任务时, SiFT 首先将词嵌入向量归一化为随机向量, 然后对归一化的嵌入向量施加扰动。我们发现归一化显著提高了微调模型的性能。对于较大的 DeBERTa 模型, 这种改进更为显著。请注意, 在我们的实验中, 我们仅在 SuperGLUE 任务上对 DeBERTa<sub>1.5B</sub> 应用 SiFT, 我们将在未来的工作中对 SiFT 进行更全面的研究。

## 5 实验

本节报告了 DeBERTa 在各种自然语言理解任务上的结果。

### 5.1 自然语言理解任务的主要成果

继之前对预训练语言模型的研究之后, 我们报告了使用大型和基础模型所取得的结果。

#### 5.1.1 大型模型的性能

| 模型                                | QQP  | MNLI | STS-B     | MRPC | CoLA | RTE  | Acc  | Acc  | Acc   | 平均值 |
|-----------------------------------|------|------|-----------|------|------|------|------|------|-------|-----|
| BERTlarge                         | 60.6 | 91.3 | 86.6/-    | 93.2 | 90.0 | 92.3 | 70.4 | 88.0 | 84.05 |     |
| RoBERTalarge                      | 68.0 | 92.2 | 90.2/90.2 | 96.4 | 92.4 | 93.9 | 86.6 | 90.9 | 88.82 |     |
| XLNetlarge                        | 69.0 | 92.3 | 90.8/90.8 | 97.0 | 92.5 | 94.9 | 85.9 | 90.8 | 89.15 |     |
| ELECTRALarge (无准确对应的中文词汇, 直接原词输出) | 69.9 | 92.4 | 90.8/90.8 | 96.9 | 92.6 | 95.0 | 88.0 | 90.8 | 89.46 |     |
| DeBERTalarge                      | 70.5 | 92.3 | 91.1/91.1 | 96.8 | 92.8 | 95.3 | 88.3 | 91.9 | 90.00 |     |

表 1: 在 GLUE 开发集上的对比结果。

我们按照 BERT (Devlin 等人, 2019 年) 的设置对大型模型进行预训练, 不过我们使用的是 Radford 等人 (2019 年) 和 Liu 等人 (2019 年 c) 的 BPE 词汇表。训练数据方面, 我们使用了维基百科 (英文维基百科 dump<sup>3</sup>; 12GB)、BookCorpus (Zhu 等人, 2015 年) (6GB)、OPENWEBTEXT (公共 Reddit 内容 (Gokaslan 和 Cohen, 2019 年); 38GB) 以及 STORIES (CommonCrawl 的一个子集 (Trinh 和 Le, 2018 年); 31GB)。经过数据去重 (Shoeybi 等人, 2019 年) 后, 总数据量约为 78GB。有关预训练数据集的详细描述, 请参阅附录 A.2。

我们使用 6 台 DGX-2 机器 (共 96 块 V100 GPU) 来训练模型。单个模型以 2K 的批次大小和 100 万步进行训练大约需要 20 天。有关详细的超参数, 请参阅附录 A。

我们在表 1 中总结了 DeBERTa 在 GLUE (Wang 等人, 2019b) 的 8 个自然语言理解任务上的结果, 其中将 DeBERTa 与结构相似的先前基于 Transformer 的预训练语言模型 (即 24 层, 隐藏层大小为 1024) 进行了比较, 包括 BERT、RoBERTa、XLNet、ALBERT 和 ELECTRA。请注意, RoBERTa、XLNet 和 ELECTRA 是在 160G 的训练数据上进行预训练的, 而 DeBERTa 是在 78G 的训练数据上进行预训练的。RoBERTa 和 XLNet 分别进行了 50 万步的预训练, 每步 8000 个样本, 相当于 40 亿个训练样本。DeBERTa 进行了 100 万步的预训练, 每步 2000 个样本, 相当于 20 亿个训练样本, 大约是 RoBERTa 或 XLNet 的一半。表 1 显示, 与 BERT 和 RoBERTa 相比, DeBERTa 在所有任务上的表现都更出色。同时, DeBERTa 在 8 个任务中有 6 个任务的表现优于 XLNet。特别是在 MRPC (比 XLNet 高 1.1%, 比 RoBERTa 高 1.0%)、RTE (比 XLNet 高 2.4%, 比 RoBERTa 高 1.7%) 和 CoLA (比 XLNet 高 1.5%, 比 RoBERTa 高 2.5%) 上的提升较为显著。在平均 GLUE 分数方面, DeBERTa 也优于其他最先进的预训练语言模型, 即 ELECTRA<sub>large</sub> 和 XLNetlarge。

在所有 GLUE 任务中, MNLI 最常被用作监测预训练语言模型 (PLM) 研究进展的指示性任务。DeBERTa 在 MNLI 上的表现显著优于所有规模相似的现有预训练语言模型, 并创造了新的最佳水平。

<sup>3</sup><https://dumps.wikimedia.org/enwiki/>

| 模型              | MNLI-m/mm SQuAD v1.1 SQuAD v2.0 RACE ReCoRD SWAG 命名实体识别 F1 | 一级方程式/电动方程式 | 一级方程式/电动方程式 | 一级方程式/电动方程式 | 一级方程式/电动方程式    | 一级方程式/电动方程式 | 一级方程式/电动方程式 |
|-----------------|------------------------------------------------------------|-------------|-------------|-------------|----------------|-------------|-------------|
| BERTlarge       | 86.6/ -                                                    | 90.9/84.1   | 81.8/79.0   | 72.0        | -              | 86.6        | 92.8        |
| ALBERTlarge     | 86.5/ -                                                    | 91.8/85.2   | 84.9/81.8   | 75.2        | -              | -           | -           |
| RoBERTalarge    | 90.2/90.2                                                  | 94.6/88.9   | 89.4/86.5   |             | 83.2 90.6/90.0 | 89.9        | 93.4        |
| XLNetlarge      | 90.8/90.8                                                  | 95.1/89.7   | 90.6/87.9   | 85.4        | -              | -           | -           |
| Megatron336M    | 89.7/90.0                                                  | 94.2/88.0   | 88.1/84.8   | 83.0        | -              | -           | -           |
| DeBERTalarge    | 91.1/91.1                                                  | 95.5/90.1   | 90.7/88.0   |             | 86.8 91.4/91.0 | 90.8        | 93.8        |
| ALBERTxxlarge   | 90.8/ -                                                    | 94.8/89.3   | 90.2/87.4   | 86.5        | -              | -           | -           |
| 梅加托恩 13 亿       | 90.9/91.0                                                  | 94.9/89.1   | 90.2/87.1   | 87.3        | -              | -           | -           |
| “Megatron 3.9B” | 91.4/91.4                                                  | 95.5/90.0   | 91.2/88.5   | 89.5        | -              | -           | -           |

表 2: 在 MNLI 领域内/外、SQuAD v1.1、SQuAD v2.0、RACE、ReCoRD、SWAG、CoNLL 2003 命名实体识别开发集上的结果。请注意，文献中缺失的结果用“-”表示。

除了 GLUE 之外，DeBERTa 还在三个类别的自然语言理解基准测试中进行了评估：（1）问答：SQuAD v1.1 (Rajpurkar 等人, 2016 年)、SQuAD v2.0 (Rajpurkar 等人, 2018 年)、RACE (Lai 等人, 2017 年)、ReCoRD (Zhang 等人, 2018 年) 和 SWAG (Zellers 等人, 2018 年)；（2）自然语言推理：MNLI (Williams 等人, 2018 年)；（3）命名实体识别：CoNLL-2003。为了进行比较，我们纳入了 ALBERT<sub>xxlarge</sub> (Lan 等人, 2019 年)<sup>4</sup> 和 Megatron (Shoeybi 等人, 2019 年)，其中 Megatron 有三种不同的模型大小，分别表示为 Megatron336M、Megatron1.3B 和 Megatron3.9B，它们均使用与 RoBERTa 相同的数据集进行训练。请注意，Megatron<sub>336M</sub> 的模型大小与其他上述模型类似<sup>5</sup>。

我们将结果总结在表 2 中。与之前具有相似模型规模的 SOTA 预训练语言模型（即 BERT、RoBERTa、XLNet、ALBERT<sub>large</sub> 和 Megatron<sub>336M</sub>）相比，DeBERTa 在所有七项任务中均表现出色。以 RACE 基准为例，DeBERTa 的表现显著优于 XLNet，高出 1.4%（86.8% 对比 85.4%）。尽管 Megatron<sub>1.3B</sub> 的规模是 DeBERTa 的三倍，但在四个基准测试中的三项，DeBERTa 的表现都优于 Megatron<sub>1.3B</sub>。我们在附录 A.4 中进一步报告了 DeBERTa 在文本生成任务上的表现。

### 5.1.2 基础款性能表现

我们用于基础模型预训练的设置与大型模型的设置类似。基础模型的结构遵循 BERT 基础模型的结构，即 L “12，H “768，A “12。我们使用 4 台 DGX-2 服务器，每台配备 64 块 V100 GPU 来训练基础模型。使用 2048 的批次大小完成 100 万步的单次预训练需要 10 天。我们使用相同的 78G 数据集训练 DeBERTa，并将其与在 160G 文本数据上训练的 RoBERTa 和 XLNet 进行比较。

我们在表 3 中总结了基础模型的结果。在所有三项任务中，DeBERTa 始终大幅优于 RoBERTa 和 XLNet，其优势幅度大于大型模型中的表现。例如，在 MNLI-m 上，DeBERTa<sub>base</sub> 的准确率比 RoBERTa<sub>base</sub> 高出 1.2%（88.8% 对 87.6%），比 XLNet<sub>base</sub> 高出 2%（88.8% 对 86.8%）。

| 模型          | MNLI-m/mm (准确率) | SQuAD v1.1 (F1/EM) | SQuAD 2.0 版 (F1/EM) |
|-------------|-----------------|--------------------|---------------------|
| RoBERTabase | 87.6/ -         | 91.5/84.6          | 83.7/80.5           |
| XLNetbase   | 86.8/ -         | -/-                | 负八十分之二              |
| DeBERTabase | 88.8/88.5       | 93.1/87.2          | 86.2/83.1           |

表 3: MNLI 领域内/领域外 (m/mm)、SQuAD v1.1 和 v2.0 开发集的结果。

<sup>4</sup>The hidden dimension of ALBERT<sub>xxlarge</sub> is 4 times of DeBERTa and the computation cost is about 4 times of DeBERTa.

<sup>5</sup>T5 (Raffel et al., 2020) has more parameters (11B). Raffel et al. (2020) only report the test results of T5 which are not comparable with other models.

## 5.2 模型分析

在本节中，我们首先进行消融实验，以量化 DeBERTa 中引入的不同组件的相对贡献。然后，我们研究收敛特性以表征模型训练效率。我们使用基础模型设置进行实验分析：在配备 16 块 V-100 GPU 的 DGX-2 机器上，使用维基百科 + Bookcorpus 数据集对模型进行 100 万步的预训练，批处理大小为 256，耗时 7 天。由于篇幅限制，我们在附录 A.7 中展示了 DeBERTa 和 RoBERTa 的不同注意力模式。

### 5.2.1 消融研究

为了验证我们的实验设置，我们从头开始对 RoBERTa 基础模型进行预训练。重新预训练的 RoBERTa 模型标记为 RoBERTa-ReImp<sub>base</sub>。为了探究 DeBERTa 中不同组件的相对贡献，我们开发了三个变体：

- -EMD 是未采用 EMD 的 DeBERTa 基础模型。
- -C2P 是去掉内容到位置项（即公式 4 中的 (c) 项）的 DeBERTa 基础模型。
- -P2C 是去掉位置到内容项（公式 4 中的 (b) 项）的 DeBERTa 基础模型。由于 XLNet 也使用相对位置偏差，因此该模型与 XLNet 加上 EMD 相近。

| 模型                       | MNLI-m/mm 准确率 | SQuAD v1.1 F1/EM 分数 | SQuAD v2.0 F1/EM 分数 | RACE A cc: RACE 准确率 |
|--------------------------|---------------|---------------------|---------------------|---------------------|
| BERTbase (德夫林等人, 2019 年) | 84.3/84.7     | 88.5/81.0           | 76.3/73.7           | 65.0                |
| 罗伯塔·刘等人 (2019 年 c)       | 84.7/ -       | 90.6/ -             | 79.7/ -             | 65.6                |
| XLNetbase 杨等人 (2019 年)   | 85.8/85.4     | -/-                 | 81.3/78.5           | 66.7                |
| RoBERTa-ReImpbase        | 84.9/85.1     | 91.1/84.8           | 79.5/76.0           | 66.8                |
| DeBERTabase              | 86.3/86.2     | 92.1/86.1           | 82.5/79.3           | 71.7                |
| -电磁驱动 (EMD)              | 86.1/86.1     | 91.8/85.8           | 81.3/78.0           | 70.3                |
| -C2P (消费者到生产商)           | 85.9/85.7     | 91.6/85.8           | 81.3/78.3           | 69.3                |
| 点对点 (P2P)                | 86.0/85.8     | 91.7/85.7           | 80.8/77.6           | 69.6                |
| - (EMD + C2P)            | 85.8/85.9     | 91.5/85.3           | 80.3/77.2           | 68.1                |
| - (EMD+P2C)              | 85.8/85.8     | 91.3/85.1           | 80.2/77.1           | 68.5                |

表 4: DeBERTa 基础模型的消融研究。

表 4 总结了在四个基准数据集上的结果。首先，RoBERTa-ReImp 在所有基准数据集上的表现与 RoBERTa 相似，这验证了我们的设置是合理的。其次，我们发现移除 DeBERTa 中的任何一个组件都会导致性能大幅下降。例如，移除 EMD (-EMD) 在 RACE 上的性能下降了 1.4%（从 71.7% 降至 70.3%），在 SQuAD v1.1 上下降了 0.3%（从 92.1% 降至 91.8%），在 SQuAD v2.0 上下降了 1.2%（从 82.5% 降至 81.3%），在 MNLI-m/mm 上分别下降了 0.2%（从 86.3% 降至 86.1%）和 0.1%（从 86.2% 降至 86.1%）。同样，移除内容到位置或位置到内容的映射在所有基准测试中都会导致性能下降。不出所料，移除两个组件会导致性能损失更大。

### 5.3 扩展至 15 亿参数

更大的预训练模型已显示出更好的泛化结果 (Raffel 等人, 2020 年; Brown 等人, 2020 年; Shoenberger 等人, 2019 年)。因此，我们构建了一个拥有 15 亿参数的更大版本的 DeBERTa，记作 DeBERTa<sub>1.5B</sub>。该模型由 48 层组成，隐藏层大小为 1536，注意力头数为 24<sup>6</sup>。DeBERTa<sub>1.5B</sub> 是在约 160G 的预训练数据集上进行训练的，与 Liu 等人 (2019c) 的数据集规模相似，并使用该数据集构建了一个大小为 128K 的新词汇表。

为了训练 DeBERTa<sub>1.5B</sub>，我们对模型架构进行了如下优化。首先，在所有注意力层中，我们将相对位置嵌入的投影矩阵  $W_k, r, W_q, r$  分别与  $W_k, c, W_q, c$  共享，以减少模型参数的数量。我们在表 13 中针对基础模型进行的消融研究表明，投影矩阵共享在保持模型性能的同时减小了模型的规模。

<sup>6</sup> See Table 8 in Appendix for the model hyperparameters.



其次，在第一个 Transformer 层旁边添加了一个卷积层，以引入子词编码的  $n$  元语法知识，并且将它们的输出相加，然后再输入到下一个 Transformer 层<sup>7</sup>中。

表 5 报告了 SuperGLUE (Wang 等人, 2019a) 的测试结果, SuperGLUE 是最流行的自然语言理解基准测试之一。SuperGLUE 包含了广泛的自然语言理解任务, 包括问答 (Clark 等人, 2019; Khashabi 等人, 2018; Zhang 等人, 2018)、自然语言推理 (Dagan 等人, 2006; Bar-Haim 等人, 2006; Giampiccolo 等人, 2007; Bentivogli 等人, 2009)、词义消歧 (Pilehvar 和 Camacho-Collados, 2019) 以及推理 (Levesque 等人, 2011; Roemmele 等人, 2011)。自 2019 年发布以来, 世界各地的顶尖研究团队一直在开发大规模的预训练语言模型, 这些模型在 SuperGLUE 上取得了显著的性能提升。

由于将 DeBERTa 扩展为更大规模的模型, 其性能得到了显著提升。截至 2020 年 12 月 29 日, 单个 DeBERTa<sub>1.5B</sub> 模型在 SuperGLUE 测试中的宏观平均得分首次超越人类表现 (89.9 对 89.8)。截至 2021 年 1 月 6 日, 集成的 DeBERTa 模型 (DeBERTa<sub>Ensemble</sub>) 在 SuperGLUE 基准测试中排名领先, 大幅超越人类基准 (90.3 对 89.8)。与拥有 110 亿参数的 T5 相比, 仅含 15 亿参数的 DeBERTa 在训练和维护方面能耗更低, 且更易于压缩和部署到各种应用程序中。

| 模型                     | 布尔 Q 准确率 | CB F1/加速  | Acc   | COPA 多重阅读理解 F1a/EM | 记录式问答 F1a/EM | 记录式问答 F1a/EM | 记录式问答 F1a/EM | 记录式问答 F1a/EM | 分数   |
|------------------------|----------|-----------|-------|--------------------|--------------|--------------|--------------|--------------|------|
| RoBERTaLarge           | 87.1     | 90.5/95.2 | 90.6  | 84.4/52.5          | 90.6/90.0    | 88.2         | 69.9         | 89.0         | 84.6 |
| NEXHA-Plus             | 87.8     | 94.4/96.0 | 93.6  | 84.6/55.1          | 90.1/89.6    | 89.1         | 74.6         | 93.2         | 86.7 |
| T511B                  | 91.2     | 93.9/96.8 | 94.8  | 88.1/63.3          | 94.1/93.4    | 92.5         | 76.9         | 93.8         | 89.3 |
| T511B + 米娜             | 91.3     | 95.8/97.6 | 97.4  | 88.3/63.0          | 94.2/93.5    | 92.7         | 77.9         | 95.9         | 90.2 |
| 人类                     | 89.0     | 95.8/98.9 | 100.0 | 81.8/51.9          | 91.7/91.3    | 93.6         | 80.0 -       | 100.0        | 89.8 |
| DeBERTa150 亿参数模型 + SIB | 90.4     | 94.9/97.2 | 96.8  | 88.2/63.7          | 94.5/94.1    | 93.2         | 76.4         | 95.9         | 89.9 |
| DeBERTaEnsemble        | 90.4     | 95.7/97.6 | 98.4  | 88.2/63.7          | 94.5/94.1    | 93.2         | 77.5         | 95.9         | 90.3 |

表 5: 使用 SuperGLUE 评估服务器对测试集进行评分的 SuperGLUE 测试集结果。所有结果均于 2021 年 1 月 6 日从 <https://super.gluebenchmark.com> 获取。

## 6 结论

本文提出了一种新的模型架构 DeBERTa (解码增强型带解耦注意力机制的 BERT), 该架构通过两种新颖的技术改进了 BERT 和 RoBERTa 模型。第一种是解耦注意力机制, 其中每个词都用两个向量分别表示其内容和位置, 词与词之间的注意力权重分别通过基于内容和相对位置的解耦矩阵计算得出。第二种是增强型掩码解码器, 它在解码层中引入绝对位置信息, 以在模型预训练阶段预测被掩码的标记。此外, 在微调过程中还使用了一种新的虚拟对抗训练方法, 以提高模型在下游任务中的泛化能力。

我们通过一项全面的实证研究证明, 这些技术显著提高了模型预训练的效率以及下游任务的表现。拥有 15 亿参数的 DeBERTa 模型在 SuperGLUE 基准测试中的宏观平均得分首次超越了人类水平。

DeBERTa 在 SuperGLUE 上超越人类表现标志着迈向通用人工智能的重要里程碑。尽管在 SuperGLUE 上取得了令人鼓舞的成绩, 但该模型远未达到自然语言理解的人类智能水平。人类极其擅长利用从不同任务中学到的知识, 无需或只需少量特定任务的演示就能解决新任务。这被称为组合泛化, 即能够将熟悉的组成部分 (子任务或基本问题解决技能) 以新的组合形式 (新任务) 进行泛化。展望未来, 值得探索如何让 DeBERTa 更明确地融入组合结构, 从而能够像人类一样将神经计算和符号计算相结合来处理自然语言。

<sup>7</sup>Please refer to Table 12 in Appendix A.6 for the ablation study of different model sizes, and Table 13 in Appendix A.6 for the ablation study of new modifications.

## 7 致谢

我们感谢 Jade Huang 和 Nikos Karampatziakis 对论文的校对以及提出的宝贵意见。我们感谢 Yoyo Liang、Saksham Singhal、Xia Song 和 Saurabh Tiwary 在大规模模型训练方面给予的帮助。我们还要感谢匿名审稿人提出的宝贵意见。

## 参考文献

- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Roy Bar-Haim, Ido Dagan, Bill Dolan, Lisa Ferro, and Danilo Giampiccolo. The second PASCAL recognising textual entailment challenge. In *Proceedings of the Second PASCAL Challenges Workshop on Recognising Textual Entailment*, 01 2006.
- Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.
- Luisa Bentivogli, Ido Dagan, Hoa Trang Dang, Danilo Giampiccolo, and Bernardo Magnini. The fifth pascal recognizing textual entailment challenge. In *In Proc Text Analysis Conference (TAC’09)*, 2009.
- Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. Semeval-2017 task 1: Semantic textual similarity-multilingual and cross-lingual focused evaluation. *arXiv preprint arXiv:1708.00055*, 2017.
- Kezhen Chen, Qiuyuan Huang, Hamid Palangi, Paul Smolensky, Kenneth D Forbus, and Jianfeng Gao. Natural-to formal-language generation using tensor product representations. *arXiv preprint arXiv:1910.02339*, 2019.
- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.
- Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of NAACL-HLT 2019*, 2019.
- Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. ELECTRA: Pre-training text encoders as discriminators rather than generators. In *ICLR*, 2020.
- Ido Dagan, Oren Glickman, and Bernardo Magnini. The pascal recognising textual entailment challenge. In *Proceedings of the First International Conference on Machine Learning Challenges: Evaluating Predictive Uncertainty Visual Object Classification, and Recognizing Textual Entailment, MLCW’ 05*, Berlin, Heidelberg, 2006.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 2978–2988, 2019.
- Marie-Catherine De Marneffe, Mandy Simons, and Judith Tonhauser. The commitmentbank: Investigating projection in naturally occurring discourse. In *proceedings of Sinn und Bedeutung*, volume 23, pp. 107–124, 2019.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186, 2019.

- William B Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*, 2005.
- Li Dong, Nan Yang, Wenhui Wang, Furu Wei, Xiaodong Liu, Yu Wang, Jianfeng Gao, Ming Zhou, and Hsiao-Wuen Hon. Unified language model pre-training for natural language understanding and generation. In *Advances in Neural Information Processing Systems*, pp. 13042–13054, 2019.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv preprint arXiv:2101.03961*, 2021.
- Danilo Giampiccolo, Bernardo Magnini, Ido Dagan, and Bill Dolan. The third PASCAL recognizing textual entailment challenge. In *Proceedings of the ACL-PASCAL Workshop on Textual Entailment and Paraphrasing*, pp. 1–9, Prague, June 2007. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/W07-1401>.
- Aaron Gokaslan and Vanya Cohen. Openwebtext corpus. <http://Skylion007.github.io>, 2019.
- Pengcheng He, Xiaodong Liu, Weizhu Chen, and Jianfeng Gao. A hybrid neural network model for commonsense reasoning. *arXiv preprint arXiv:1907.11983*, 2019a.
- Pengcheng He, Yi Mao, Kaushik Chakrabarti, and Weizhu Chen. X-sql: reinforce schema representation with context. *arXiv preprint arXiv:1908.08113*, 2019b.
- Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Ian Simon, Curtis Hawthorne, Noam Shazeer, Andrew M Dai, Matthew D Hoffman, Monica Dinculescu, and Douglas Eck. Music transformer: Generating music with long-term structure. 2018.
- Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Tuo Zhao. SMART: Robust and efficient fine-tuning for pre-trained natural language models through principled regularized optimization. In *ACL*, July 2020. doi: 10.18653/v1/2020.acl-main.197.
- Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S Weld, Luke Zettlemoyer, and Omer Levy. Spanbert: Improving pre-training by representing and predicting spans. *Transactions of the Association for Computational Linguistics*, 8:64–77, 2020.
- Kamal Raj Kanakarajan, Bhuvana Kundumani, and Malaikannan Sankarasubbu. Small-bench nlp: Benchmark for small single gpu trained models in natural language processing. *ArXiv*, abs/2109.10847, 2021.
- Daniel Khashabi, Snigdha Chaturvedi, Michael Roth, Shyam Upadhyay, and Dan Roth. Looking beyond the surface: A challenge set for reading comprehension over multiple sentences. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pp. 252–262, 2018.
- Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2019.
- Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. Race: Large-scale reading comprehension dataset from examinations. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pp. 785–794, 2017.
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations. In *International Conference on Learning Representations*, 2019.
- Hector Levesque, Ernest Davis, and Leora Morgenstern. The winograd schema challenge. In *Thirteenth International Conference on the Principles of Knowledge Representation and Reasoning*, 2012.

- Hector J Levesque, Ernest Davis, and Leora Morgenstern. The Winograd schema challenge. In *AAAI Spring Symposium: Logical Formalizations of Commonsense Reasoning*, volume 46, pp. 47, 2011.
- Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. On the variance of the adaptive learning rate and beyond. In *International Conference on Learning Representations*, 2019a.
- Xiaodong Liu, Pengcheng He, Weizhu Chen, and Jianfeng Gao. Multi-task deep neural networks for natural language understanding. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4487–4496, Florence, Italy, July 2019b. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/P19-1441>.
- Xiaodong Liu, Hao Cheng, Pengcheng He, Weizhu Chen, Yu Wang, Hoifung Poon, and Jianfeng Gao. Adversarial training for large neural language models. *arXiv preprint arXiv:2004.08994*, 2020.
- Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019c.
- Ilya Loshchilov and Frank Hutter. Fixing weight decay regularization in adam. 2018.
- Yu Meng, Chenyan Xiong, Payal Bajaj, Saurabh Tiwary, Paul Bennett, Jiawei Han, and Xia Song. Coco-lm: Correcting and contrasting text sequences for language model pretraining. 2021.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv*, pp. arXiv–1609, 2016.
- Shervin Minaee, Nal Kalchbrenner, Erik Cambria, Narjes Nikzad, Meysam Chenaghlu, and Jianfeng Gao. Deep learning based text classification: A comprehensive review. *arXiv preprint arXiv:2004.03705*, 2020.
- Takeru Miyato, Shin-ichi Maeda, Masanori Koyama, and Shin Ishii. Virtual adversarial training: a regularization method for supervised and semi-supervised learning. *IEEE transactions on pattern analysis and machine intelligence*, 41(8):1979–1993, 2018.
- Mohammad Taher Pilehvar and Jose Camacho-Collados. Wic: the word-in-context dataset for evaluating context-sensitive meaning representations. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 1267–1273, 2019.
- Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8), 2019.
- Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020. URL <http://jmlr.org/papers/v21/20-074.html>.
- Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, November 2016.
- Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for squad. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 784–789, 2018.
- Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S. Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *2011 AAAI Spring Symposium Series*, 2011.
- Imanol Schlag, Paul Smolensky, Roland Fernandez, Nebojsa Jojic, Jürgen Schmidhuber, and Jianfeng Gao. Enhancing the transformer with explicit relational encoding for math problem solving. *arXiv preprint arXiv:1910.06611*, 2019.

- Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pp. 464–468, 2018.
- Tao Shen, Yi Mao, Pengcheng He, Guodong Long, Adam Trischler, and Weizhu Chen. Exploiting structured knowledge in text via graph-guided representation learning. *arXiv preprint arXiv:2004.14224*, 2020.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using gpu model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Paul Smolensky. Tensor product variable binding and the representation of symbolic structures in connectionist systems. *Artificial intelligence*, 46(1-2):159–216, 1990.
- Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pp. 1631–1642, 2013.
- Yu Sun, Shuohuan Wang, Yukun Li, Shikun Feng, Xuyi Chen, Han Zhang, Xin Tian, Danxiang Zhu, Hao Tian, and Hua Wu. Ernie: Enhanced representation through knowledge integration. *arXiv preprint arXiv:1904.09223*, 2019.
- Trieu H Trinh and Quoc V Le. A simple method for commonsense reasoning. *arXiv preprint arXiv:1806.02847*, 2018.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pp. 5998–6008, 2017.
- Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems. In *Advances in neural information processing systems*, pp. 3266–3280, 2019a.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *7th International Conference on Learning Representations, ICLR 2019*, 2019b.
- Wei Wang, Bin Bi, Ming Yan, Chen Wu, Zuyi Bao, Liwei Peng, and Luo Si. Structbert: Incorporating language structures into pre-training for deep language understanding. *arXiv preprint arXiv:1908.04577*, 2019c.
- Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*, 2018.
- Adina Williams, Nikita Nangia, and Samuel Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pp. 1112–1122. Association for Computational Linguistics, 2018. URL <http://aclweb.org/anthology/N18-1101>.
- Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Russ R Salakhutdinov, and Quoc V Le. Xlnet: Generalized autoregressive pretraining for language understanding. In *Advances in neural information processing systems*, pp. 5754–5764, 2019.
- Rowan Zellers, Yonatan Bisk, Roy Schwartz, and Yejin Choi. Swag: A large-scale adversarial dataset for grounded commonsense inference. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 93–104, 2018.

Sheng Zhang, Xiaodong Liu, Jingjing Liu, Jianfeng Gao, Kevin Duh, and Benjamin Van Durme. ReCoRD: Bridging the gap between human and machine commonsense reading comprehension. *arXiv preprint 1810.12885*, 2018.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *Proceedings of the IEEE international conference on computer vision*, pp. 19–27, 2015.

## 附录

### A.1 数据集

| 语料库             | 任务               | #火车     | #开发    | #测试    | #标签 | 指标            |
|-----------------|------------------|---------|--------|--------|-----|---------------|
| 通用语言理解评估 (GLUE) |                  |         |        |        |     |               |
| CoLA            | 可接受性             | 850     | 一千     | 一千     | 2   | 马修斯系数         |
| SST             | 情绪               | 六万七千    | 872    | 180    | 2   | 准确性           |
| MNLI            | 自然语言推理           | 39.3 万  | 两万     | 两万     | 3   | 准确性           |
| RTE             | 自然语言推理           | 250     | 276    | 三千     | 2   | 准确性           |
| WNLI            | 自然语言推理           | 634     | 71     | 146    | 2   | 准确性           |
| QQP             | 释义; 改述; 换言之; 换说法 | 304 千字节 | 四万 (元) | 39.1 万 | 2   | 准确率/F1 值      |
| MRPC            | 释义; 改述; 同义表述     | 三千七百    | 408    | 170    | 2   | 准确率/F1 值      |
| QNLI            | 问答/自然语言推理        | 108 千字节 | 五千七百   | 五千七百   | 2   | 准确性           |
| STS-B           | 相似性              | 700     | 150    | 140    | 1   | 皮尔逊/斯皮尔曼相关系数  |
| SuperGLUE       |                  |         |        |        |     |               |
| WSC             | 共指               | 554 千字节 | 104    | 146    | 2   | 准确性           |
| 布尔问题            | 问答               | 9,427   | 3,270  | 3,245  | 2   | 准确性           |
| 科帕              | 问答               | 40 万    | 100    | 500    | 2   | 准确性           |
| CB              | 自然语言推理           | 250     | 57     | 250    | 3   | 准确率/F1 值      |
| RTE             | 自然语言推理           | 250     | 276    | 三千     | 2   | 准确性           |
| WiC             | WSD              | 250     | 276    | 三千     | 2   | 准确性           |
| ReCoRD          | MRC              | 10.1 万  | 10 千米  | 10 千米  | -   | 精确匹配 (EM) /F1 |
| MultiRC         | 多项选择题            | 5,100   | 953    | 1,800  | -   | 精确匹配 (EM) /F1 |
| 问答系统            |                  |         |        |        |     |               |
| SQuAD 1.1 版     | MRC              | 8760    | 1050   | 950    | -   | 精确匹配 (EM) /F1 |
| SQuAD 2.0 版     | MRC              | 13030   | 1190   | 890    | -   | 精确匹配 (EM) /F1 |
| 赛跑              | MRC              | 87,866  | 4,887  | 4,934  | 4   | 准确性           |
| SWAG            | 多项选择题            | 七万三千五百  | 两万     | 两万     | 4   | 准确性           |
| 标记分类            |                  |         |        |        |     |               |
| CoNLL 2003      | 命名实体识别           | 14,987  | 3,466  | 3,684  | 8   | F1            |

表 6: 自然语言处理应用基准的汇总信息。

GLUE (通用语言理解评估) 基准测试包含九项自然语言理解 (NLU) 任务。如表 6 所示, 这些任务包括问答 (Rajpurkar 等人, 2016 年)、语言可接受性 (Warstadt 等人, 2018 年)、情感分析 (Socher 等人, 2013 年)、文本相似度 (Cer 等人, 2017 年)、句意相似性检测 (Dolan 和 Brockett, 2005 年) 以及自然语言推理 (NLI) (Dagan 等人, 2006 年; Bar-Haim 等人, 2006 年; Giampiccolo 等人, 2007 年; Bentivogli 等人, 2009 年; Levesque 等人, 2012 年; Williams 等人, 2018 年)。这些任务的多样性使得 GLUE 非常适合用于评估 NLU 模型的泛化能力和鲁棒性。

SuperGLUE 是 GLUE 基准测试的延伸, 但难度更高, 它包含八项自然语言理解任务。这些任务涵盖了多种类型, 包括问答 (Zhang 等人, 2018 年; Clark 等人, 2019 年; Khashabi 等人, 2018 年)、自然语言推理 (Dagan 等人, 2006 年; Bar-Haim 等人, 2006 年; Giampiccolo 等人, 2007 年; Bentivogli 等人, 2009 年; De Marneffe 等人, 2019 年)、共指消解 (Levesque 等人, 2012 年) 以及词义消歧 (Pilehvar 和 Camacho-Collados, 2019 年)。

RACE 是一个大规模的机器阅读理解数据集, 是从中国针对中学生和高中生的英语考试中收集而来的 (Lai 等人, 2017 年)。

斯坦福问答数据集 (SQuAD) v1.1 和 v2.0 (Rajpurkar 等人, 2016 年; 2018 年) 是流行的机器阅读理解基准。其段落来自约 500 篇维基百科文章, 问题和答案通过众包获得。SQuAD v2.0 数据集包含了关于相同段落的无法回答的问题。

SWAG 是一个用于基于情境的常识推理任务的大规模对抗数据集，它将自然语言推理和基于物理的情境推理统一起来（Zellers 等人，2018 年）。SWAG 包含了 11.3 万个关于具体情境的多项选择题。

CoNLL 2003 是一个由多种来源的英文文本组成的语料库，其中包含 4 种类型的命名实体。

## A.2 预训练数据集

对于 DeBERTa 的预训练，我们使用了维基百科（英文维基百科 dump<sup>8</sup>；12GB）、BookCorpus（Zhu 等人，2015 年）<sup>9</sup>（6GB）、OPENWEBTEXT（公开的 Reddit 内容（Gokaslan 和 Cohen，2019 年）；38GB）和 STORIES<sup>10</sup>（CommonCrawl 的一个子集（Trinh 和 Le，2018 年）；31GB）。经过数据去重（Shoeybi 等人，2019 年）后，总数据量约为 78GB。在预训练过程中，我们还从训练数据中抽取 5% 作为验证集以监控训练过程。表 7 对不同预训练模型所使用的数据集进行了比较。

| 模型              | 维基百科+图书 | 开放网络文本 | 故事 | 新闻 | 吉尼斯世界纪录 | 常识问答 | 网络爬虫 | 110 gb |
|-----------------|---------|--------|----|----|---------|------|------|--------|
| BERT            | X       |        |    |    |         |      |      |        |
| XLNet           | X       |        |    |    |         | X    | X    | X      |
| RoBERTa         | X       | X      | X  | X  | X       |      |      |        |
| DeBERTa         | X       | X      | X  | X  |         |      |      |        |
| DeBERTa 15 亿参数版 | X       | X      | X  | X  | X       |      |      |        |

表 7: 预训练数据的比较。

## A.3 实施细节

我们沿用 RoBERTa（Liu 等人，2019c）的做法，采用动态数据分批。我们还引入了跨度掩码（Joshi 等人，2020）作为额外的掩码策略，跨度大小最多为 3。预训练的详细超参数见表 8。在预训练阶段，我们使用带有权重衰减（Loshchilov 和 Hutter，2018）的 Adam（Kingma 和 Ba，2014）作为优化器。在微调阶段，尽管在某些任务（如 CoLA、RTE 和 RACE）上使用 RAdam（Liu 等人，2019a）能获得更好的且更稳健的结果，但为了公平比较，我们仍使用 Adam（Kingma 和 Ba，2014）作为优化器。在微调阶段，我们对每个任务都采用超参数搜索过程，每次运行在 DGX-2 节点上大约需要 1 到 2 小时。所有超参数见表 9。模型选择依据是任务特定开发集上的性能。

我们的代码是基于 Huggingface Transformers<sup>11</sup>、FairSeq<sup>12</sup> 和 Megatron（Shoeybi 等人，2019 年）<sup>13</sup> 实现的。

### A.3.1 预训练效率

为了探究模型预训练的效率，我们将微调模型在下游任务上的表现作为预训练步数的函数进行绘制。如图 1 所示，对于 RoBERTa-ReImp<sub>base</sub> 和 DeBERTa<sub>base</sub>，我们每 15 万步预训练保存一次检查点，然后在两个具有代表性的下游任务 MNLI 和 SQuAD v2.0 上对检查点进行微调，并分别报告准确率和 F1 分数。作为参考，我们还报告了原始 RoBERTa<sub>base</sub>（Liu 等人，2019c）和 XLNet<sub>base</sub>（Yang 等人，2019）的最终模型性能。结果表明，在预训练过程中，DeBERTa<sub>base</sub> 一直优于 RoBERTa-ReImp<sub>base</sub>。

<sup>8</sup> <https://dumps.wikimedia.org/enwiki/>

<sup>9</sup> [https://github.com/butsugiri/homemade\\_bookcorpus](https://github.com/butsugiri/homemade_bookcorpus)

<sup>10</sup> [https://github.com/tensorflow/models/tree/master/research/lm\\_commonsense](https://github.com/tensorflow/models/tree/master/research/lm_commonsense)

<sup>11</sup> <https://github.com/huggingface/transformers>

<sup>12</sup> <https://github.com/pytorch/fairseq>

<sup>13</sup> <https://github.com/NVIDIA/Megatron-LM>



| 超参数          | DeBERTa1.5B | DeBERTalarge | DeBERTabase | DeBERTabase消融版 |
|--------------|-------------|--------------|-------------|----------------|
| 层数           | 48          | 24           | 12          | 12             |
| 隐藏层大小        | 1536        | 1024         | 768         | 768            |
| FNN 内部隐藏层大小  | 6144        | 4096         | 3072        | 3072           |
| 注意力头         | 24          | 16           | 12          | 12             |
| 注意力头大小       | 64          | 64           | 64          | 64             |
| 辍学           | 0.1         | 0.1          | 0.1         | 0.1            |
| 热身步骤         | 10 千米       | 10 千米        | 10 千米       | 10 千米          |
| 学习率          | 1.5e-4      | 2e-4         | 2e-4        | 1e-4           |
| 批量大小         | 2048        | 2048         | 2048        | 256            |
| 权重衰减         | 0.01        | 0.01         | 0.01        | 0.01           |
| 最大步数         | 1 兆 (字节)    | 1 兆 (字节)     | 1 兆 (字节)    | 1 兆 (字节)       |
| 学习率衰减        | 线性的         | 线性的          | 线性的         | 线性的            |
| 亚当           | 1e-6        | 1e-6         | 1e-6        | 1e-6           |
| 亚当 $\beta_1$ | 0.9         | 0.9          | 0.9         | 0.9            |
| 亚当 $\beta_2$ | 0.999       | 0.999        | 0.999       | 0.999          |
| 梯度裁剪         | 1.0         | 1.0          | 1.0         | 1.0            |
| DGX-2 节点数量   | 16          | 6            | 4           | 1              |
| 训练时间         | 30 天        | 20 天         | 10 天        | 7 天            |

表 8: DeBERTa 预训练的超参数。

| 超参数          | DeBERTa 15 亿参数版      | DeBERTalarge             | DeBERTabase                |
|--------------|----------------------|--------------------------|----------------------------|
| 任务层的丢弃       | {0, 0.15, 0.3}       | {0, 0.1, 0.15}           | {0, 0.1, 0.15}             |
| 热身步骤         | {50, 100, 500, 1000} | {50, 100, 500, 1000}     | {50, 100, 500, 1000}       |
| 学习率          | {1e-6, 3e-6, 5e-6}   | {5e-6, 8e-6, 9e-6, 1e-5} | {1.5e-5, 2e-5, 3e-5, 4e-5} |
| 批量大小         | {16, 32, 64}         | {16, 32, 48, 64}         | {16, 32, 48, 64}           |
| 权重衰减         | 0.01                 | 0.01                     | 0.01                       |
| 最大训练轮次       | 10                   | 10                       | 10                         |
| 学习率衰减        | 线性的                  | 线性的                      | 线性的                        |
| 亚当           | 1e-6                 | 1e-6                     | 1e-6                       |
| 亚当 $\beta_1$ | 0.9                  | 0.9                      | 0.9                        |
| 亚当 $\beta_2$ | 0.999                | 0.999                    | 0.999                      |
| 梯度裁剪         | 1.0                  | 1.0                      | 1.0                        |

表 9: 在下游任务中对 DeBERTa 进行微调的超参数。

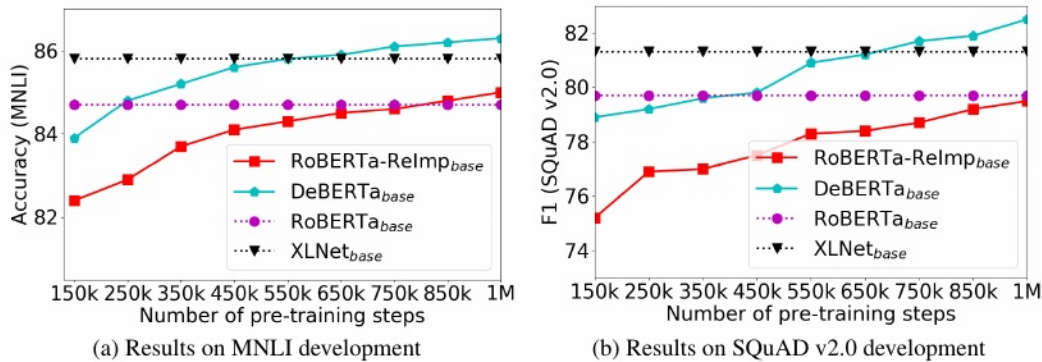


图 1: DeBERTa 及其同类模型在 MNLI 和 SQuAD v2.0 开发集上的预训练性能曲线。

#### A.4 生成任务的主要结果

除了自然语言理解任务外，DeBERTa 还可以扩展以处理自然语言生成任务。为了让 DeBERTa 能够像自回归模型那样进行文本生成，我们使用三角矩阵进行自注意力机制，并按照 Dong 等人（2019 年）的方法将自注意力掩码的上三角部分设置为 -8。

我们使用 Wikitext-103 (Merity 等人, 2016 年) 对 DeBERTa 在自回归语言模型 (ARLM) 任务上的表现进行评估。为此，我们训练了一个新的 DeBERTa 版本，记为 DeBERTa-MT。它与 UniLM (Dong 等人, 2019 年) 一样，通过掩码语言模型 (MLM) 和自回归语言模型 (ARLM) 任务联合预训练。预训练的超参数遵循 DeBERTa<sub>base</sub> 中的设置，只是训练步数较少 (20 万步)。为了进行比较，我们以 RoBERTa 作为基线，并将 GPT-2 和 Transformer-XL 作为额外的参考。DeBERTa-AP 是 DeBERTa 的一个变体，在输入层中加入了绝对位置嵌入，与 RoBERTa 类似。为了公平比较，所有这些模型都是在类似设置下预训练的基础模型。

| 模型   |      |      | RoBERTa                        | DeBERTa-AP | DeBERTa | DeBERTa-MT | GPT-2 |
|------|------|------|--------------------------------|------------|---------|------------|-------|
|      |      |      | Transformer-XL (这些是模型名称, 无需翻译) |            |         |            |       |
| 开发人员 | 21.6 | 20.7 | 20.5                           | 19.5       | -       | -          | 23.1  |
| 测试人员 | 21.6 | 20.0 | 19.9                           | 19.5       | 37.50   |            | 24    |

表 10: 语言模型在 Wikitext-103 上的困惑度结果 (数值越低越好)。

表 10 总结了在 Wikitext-103 上的结果。我们看到 DeBERTa<sub>base</sub> 在开发集和测试集上的困惑度都更低，并且使用 MLM 和 ARLM 进行联合训练进一步降低了困惑度。DeBERTa-AP 不如 DeBERTa 表明，在解码层将单词的绝对位置嵌入作为 EMD (如 DeBERTa 所做) 比在输入层 (如 RoBERTa 所做) 更有效。

#### A.5 处理长序列输入

采用相对位置偏差时，我们选择将最大相对距离截断为  $k$ ，如公式 3 所示。因此，在每一层中，每个标记最多可以直接关注  $2pk - 1q$  个标记以及自身。通过堆叠 Transformer 层，第  $l$  层中的每个标记最多可以间接关注  $p2k - 1ql$  个标记。以 DeBERTa<sub>large</sub> 为例，其中  $k = 512$ ， $L = 24$ ，理论上可以处理的最大序列长度为 24,528。这是我们的设计选择带来的一个附带好处，我们发现这对 RACE 任务很有益处。长序列对 RACE 任务的影响比较见表 11。

| 序列长度 | 中等   | 高    | 准确率  |
|------|------|------|------|
| 512  | 88.8 | 85.0 | 86.3 |
| 768  | 88.7 | 86.3 | 86.8 |

表 11: DeBERTa 处理 RACE 任务中长序列输入的效果

长序列处理是一个活跃的研究领域。已有大量研究对 Transformer 架构进行扩展以处理长序列 (Beltagy 等人, 2020 年; Kitaev 等人, 2019 年; Child 等人, 2019 年; Dai 等人, 2019 年)。我们未来的研究方向之一是将 DeBERTa 扩展以处理极长的序列。

#### A.6 不同模型规模的性能改进

在本小节中，我们研究了不同模型规模应用于大型模型对 GLUE 的影响。表 12 总结了结果，表明规模更大的模型能够取得更好的效果，并且 SiFT 也始终如一地提升了模型性能。

| 模型                      | QQP  | ANLI-m | STS-B     | MRTE | MRCC | 校对   | Acc  | Acc  | Acc   | 平均值 |
|-------------------------|------|--------|-----------|------|------|------|------|------|-------|-----|
| DeBERTaLarge            | 70.5 | 92.3   | 91.1/91.1 | 96.8 | 92.8 | 95.3 | 88.3 | 91.9 | 90.00 |     |
| DeBERTa 9 亿参数版          | 71.1 | 92.3   | 91.7/91.6 | 97.5 | 92.0 | 95.8 | 93.5 | 93.1 | 90.86 |     |
| DeBERTa 15 亿参数版         | 72.0 | 92.7   | 91.7/91.9 | 97.2 | 92.9 | 96.0 | 93.9 | 92.0 | 91.17 |     |
| DeBERTa150 亿参数模型 + SIFT | 73.5 | 93.0   | 92.0/92.1 | 97.5 | 93.2 | 96.5 | 96.5 | 93.2 | 91.93 |     |

表 12: 不同规模的 DeBERTa 模型在 GLUE 开发集上的对比结果。

| 模型                | 参数         | MNLI-m/mm 准确率 | SQuAD v1.1 F1/EM 分数 | SQuAD v2.0 F1/EM 分数 |
|-------------------|------------|---------------|---------------------|---------------------|
| RoBERTa-ReImpbase | 1.2 亿      | 84.9/85.1     | 91.1/84.8           | 79.5/76.0           |
| DeBERTabase       | 134 兆 (字节) | 86.3/86.2     | 92.1/86.1           | 82.5/79.3           |
| + 共享投影            | 120兆       | 86.3/86.3     | 92.2/86.2           | 82.3/79.5           |
| + 会议              | 122 兆 (字节) | 86.3/86.5     | 92.5/86.4           | 82.5/79.7           |
| + 128k 词汇量        | 1.9 亿      | 86.7/86.9     | 93.1/86.8           | 83.0/80.1           |

表 13: 在 DeBERTa<sub>1.5B</sub> 和 DeBERTa<sub>900M</sub> 模型中额外修改的消融研究。请注意，我们是在 DeBERTa<sub>base</sub> 的基础上逐步添加每个组件的。

## A.7 模型复杂度

借助解缠注意力机制，我们引入了三组额外的参数  $W_q, r, W_k, r \in \mathbb{R}^{d \times d}$  和  $P \in \mathbb{R}^{2k \times d}$ 。模型参数的总增加量为  $2Ld^2 + 2kd$ 。对于大型模型 ( $d = 1024, L = 24, k = 512$ )，这相当于约 4900 万个额外参数，增加了 13%。对于基础模型 ( $d = 768, L = 12, k = 512$ )，这相当于 1400 万个额外参数，增加了 12%。然而，通过在内容嵌入和位置嵌入之间共享投影矩阵，即  $W_q, r = W_q, c, W_k, r = W_k, c$ ，DeBERTa 的参数数量与 RoBERTa 相同。我们在基础模型上的实验表明，结果几乎相同，如表 13 所示。

由于额外的位置到内容和内容到位置注意力得分的计算，额外的计算复杂度为  $O(pNkdq)$ 。与 BERT 或 RoBERTa 相比，这增加了 30% 的计算成本。与同样使用相对位置嵌入的 XLNet 相比，计算成本增加了约 15%。通过融合注意力计算内核的进一步优化可以显著降低这一额外成本。对于 EMD，由于预训练中的解码器仅重构被掩码的标记，因此对于未掩码的标记不会引入额外的计算成本。在 15% 的标记被掩码且仅使用两层解码器的情况下，额外成本为  $0.15 \times 2 \times L$ ，这导致 EMD 中基础模型 ( $L = 12$ ) 的额外计算成本仅为 3%，大型模型 ( $L = 24$ ) 为 2%。

## A.8 增强型掩码解码器的更多细节

EMD 的结构如图 2b 所示。EMD 有两个输入 (即  $I$  和  $H$ )。  $H$  表示来自前一层 Transformer 的隐藏状态，而  $I$  可以是解码所需的任何必要信息，例如  $H$ 、绝对位置嵌入或前一层 EMD 的输出。 $n$  表示  $n$  个堆叠的 EMD 层，其中每个 EMD 层的输出将成为下一个 EMD 层的输入  $I$ ，而最后一个 EMD 层的输出将直接输入到语言模型头中。这  $n$  层可以共享相同的权重。在我们的实验中，我们为  $n = 2$  层共享相同的权重以减少参数数量，并将绝对位置嵌入作为第一个 EMD 层的  $I$ 。当  $I = H$  且  $n = 1$  时，EMD 与 BERT 解码器层相同。然而，EMD 更具通用性和灵活性，因为它可以采用各种类型的输入信息进行解码。

## A.9 注意模式

为了直观展示 DeBERTa 与 RoBERTa 的运作差异，我们在图 3 中呈现了 RoBERTa、DeBERTa 以及三种 DeBERTa 变体在最后一层自注意力机制中的注意力模式。

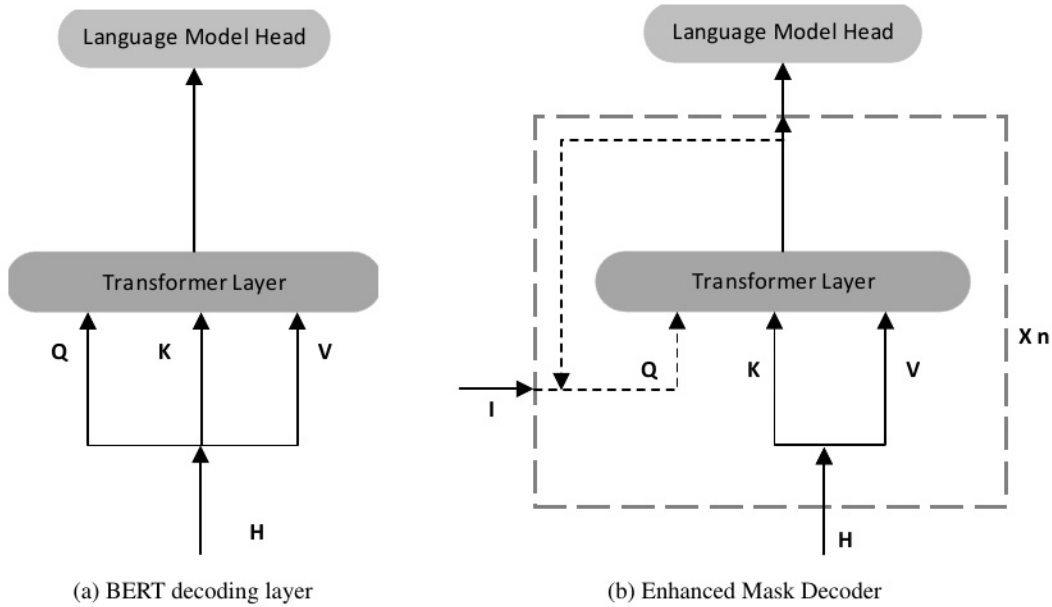


图 2: 解码层的比较。

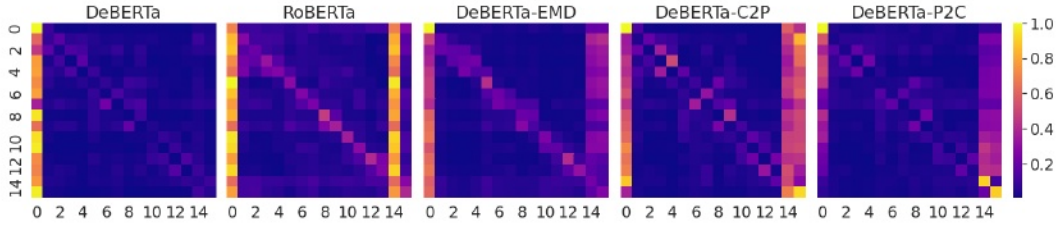


图 3: DeBERTa、RoBERTa 以及 DeBERTa 的变体（即分别去掉 EMD、C2P 和 P2C 的 DeBERTa）最后一层的注意力模式比较。

我们观察到两个差异。首先，RoBERTa 在一个标记关注自身时有明显的对角线效应。但在 DeBERTa 中这种效应不太明显。这可以归因于 EMD 的使用，在 EMD 中，绝对位置嵌入被添加到内容隐藏状态作为查询向量，这一点从 DeBERTa-EMD 的注意力模式中可以得到验证，其对角线效应比原始 DeBERTa 更明显。其次，我们注意到 RoBERTa 的注意力模式中存在垂直条纹，这主要是由高频功能词或标记（例如“a”、“the”和标点符号）引起的。对于 DeBERTa，条纹仅出现在第一列，代表 [CLS] 标记。我们推测对 [CLS] 的主要强调是有益的，因为在下游任务中 [CLS] 的特征向量通常被用作整个输入序列的上下文表示。我们还注意到，在三个 DeBERTa 变体的模式中，垂直条纹效应相当明显。

我们在图 4 和图 5 中给出了三个额外的示例，以说明 DeBERTa 和 RoBERTa 的不同注意力模式。

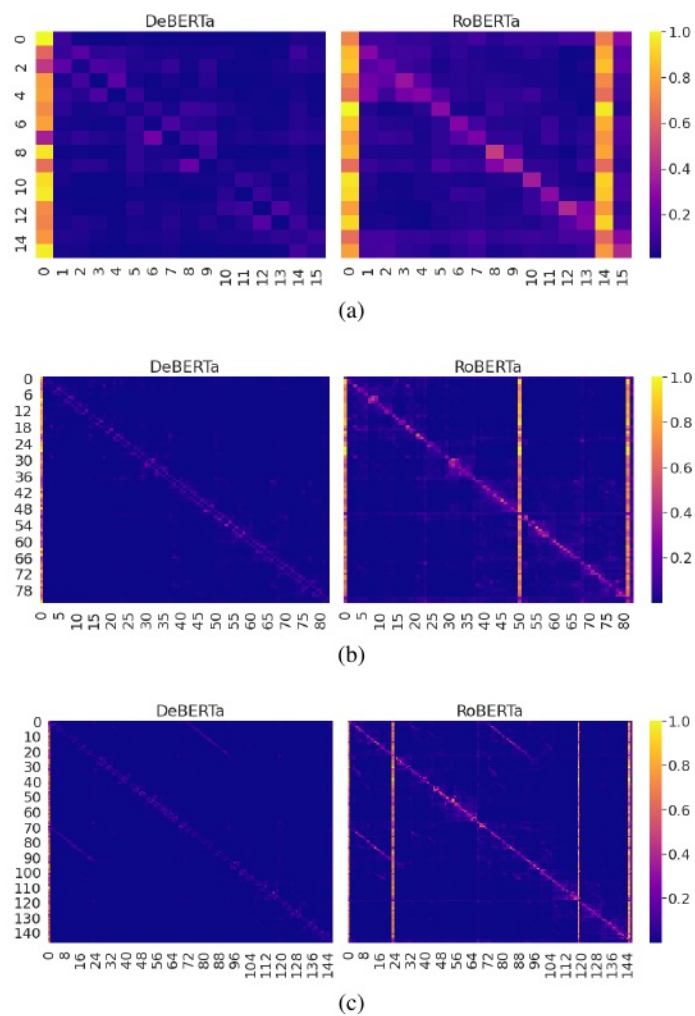


图 4: DeBERTa 和 RoBERTa 最后一层注意力模式的比较。

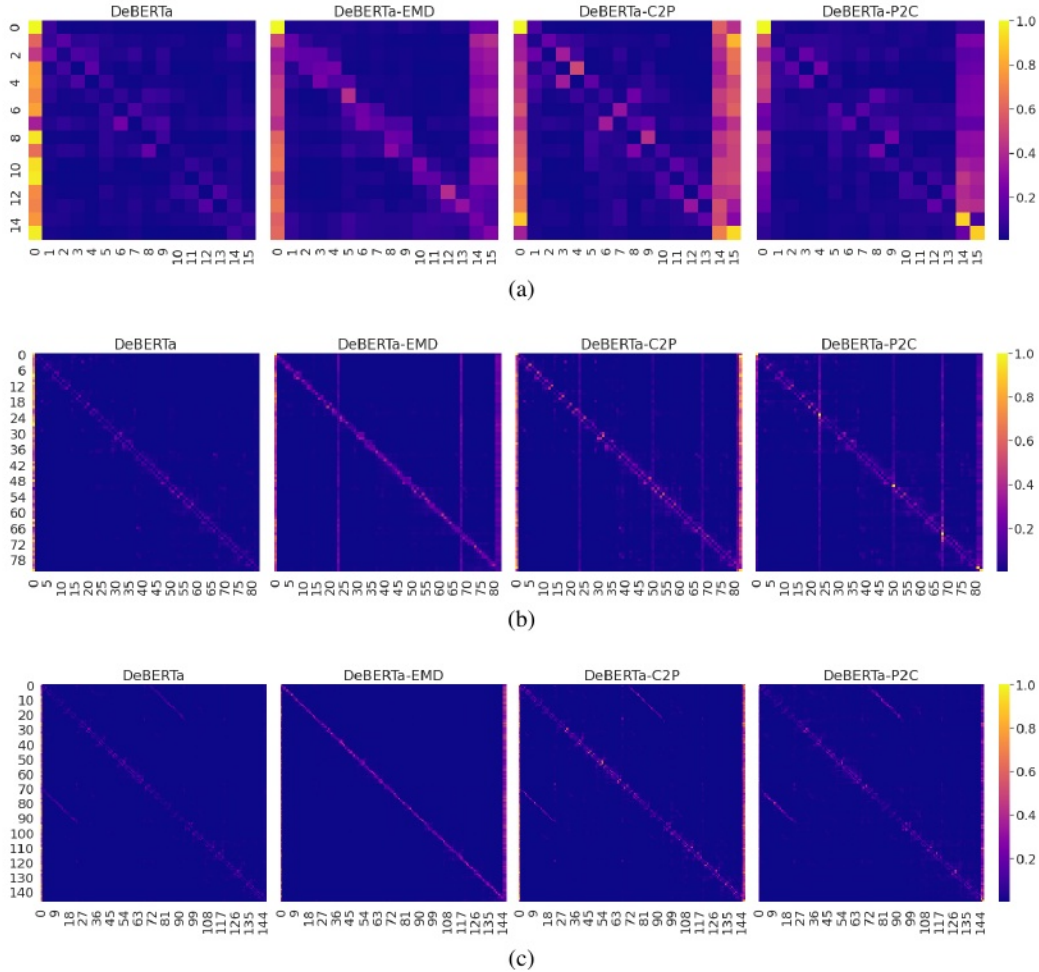


图 5: DeBERTa 及其变体（即分别去掉 EMD、C2P 和 P2C 的 DeBERTa）最后一层注意力模式的比较。

### A.10 解释微调中的差异

考虑到不同运行中微调的差异，在我们的实验中，我们始终遵循（Liu 等人，2019c）的做法，通过使用五个不同随机初始化种子的运行结果取平均值来报告下游任务的结果，并在比较结果时进行显著性检验。如表 14 所示的示例，DeBERTa<sub>base</sub> 显著优于 RoBERTa<sub>base</sub>（ $p$  值  $< 0.05$ ）。

| 模型          | MNLI 匹配（最小值/最大值/平均值）                 | SQuAD v1.1（最小值/最大值/平均值）              | p 值  |
|-------------|--------------------------------------|--------------------------------------|------|
| RoBERTabase | 84.7/85.0/84.9                       | 90.8/91.3/91.1（由于原文为数字，无需翻译，直接照搬即可。） | 0.00 |
| DeBERTabase | 86.1/86.5/86.3（由于原文为数字，无需翻译，直接照搬即可。） | 91.5/91.8/91.6（由于原文为数字，无需翻译，直接照搬即可。） | 0.01 |

表 14: DeBERTa 与 RoBERTa 在 MNLI 匹配任务和 SQuAD v1.1 上的对比。

### A.11 进一步提高模型效率

除了将参数量扩展到数十亿甚至数万亿的变压器模型（Raffel 等人，2020 年；Brown 等人，2020 年；Fedus 等人，2021 年）之外，提高模型的参数效率也十分重要（Kanakarajan 等人，2021 年）。在 A.3.1 节中，我们已经表明 DeBERTa 比 BERT 和 RoBERTa 更具参数效率。在本节中，我们将展示在参数效率方面的进一步改进。

替换标记检测 (RTD) 是 ELECTRA (Clark 等人, 2020 年) 提出的一种新的预训练目标。它已被证明比掩码语言模型 (MLM) (Devlin 等人, 2019 年; Liu 等人, 2019 年 c) 更有效。在 DeBERTa 中, 我们将 MLM 目标替换为 RTD 目标, 并将新变体标记为 DeBERTa<sub>RT D</sub>。我们使用与 DeBERTa<sub>1.5B</sub> 相同的 160GB 数据, 以小型、基础和大型设置对 DeBERTa<sub>RT D</sub> 进行预训练。遵循 (Meng 等人, 2021 年), 我们将生成器的宽度设置为与判别器相同, 但深度仅为判别器的一半。其他超参数与 DeBERTa<sub>base</sub> 或 DeBERTa<sub>large</sub> 保持一致。对于新成员 DeBERTa<sub>RT D<sub>small</sub></sub>, 它有 6 层, 宽度与 DeBERTa<sub>RT D<sub>base</sub></sub> 相同。我们在 MNLI 和 SQuAD v2 数据集上评估我们的模型。表 15 总结了结果。我们观察到 DeBERTa<sub>RT D<sub>base</sub></sub> 和 DeBERTa<sub>RT D<sub>large</sub></sub> 显著优于其他模型。例如, DeBERTa<sub>RT D<sub>large</sub></sub> 在 MNLI 和 SQuAD v2.0 上分别比 DeBERTaLarge (之前的最先进模型) 提高了 0.9 的绝对值。值得注意的是, DeBERTa<sub>RT D<sub>large</sub></sub> 与 DeBERTa1.5B 相当, 但参数仅为 DeBERTa1.5B 的 1/3。此外, DeBERTa<sub>RT D<sub>small</sub></sub> 甚至大幅优于 BERT<sub>large</sub>。所有这些都证明了 DeBERTa<sub>RT D</sub> 模型的高效性, 并清楚地表明了进一步提高模型参数效率的巨大潜力。我们的工作为未来研究参数效率更高的预训练语言模型奠定了基础。

| 模型                                            | MNLI (匹配/不匹配准确率) | SQuAD 2.0 版 (F1/EM) |
|-----------------------------------------------|------------------|---------------------|
| BERT <sub>RT D<sub>small</sub></sub>          | 88.2/87.9        | 82.9/80.4           |
| BERT <sub>base</sub>                          | 84.3/84.7        | 76.3/73.7           |
| RoBERTa <sub>base</sub>                       | 87.6/ -          | 83.7/80.5           |
| ELECTRA <sub>base</sub>                       | 88.8/ -          | 83.3/80.5           |
| DeBERTa <sub>base</sub>                       | 88.8/88.5        | 86.2/83.1           |
| DeBERTa <sub>RT D<sub>base</sub></sub>        | 90.6/90.8        | 88.4/85.4           |
| BERT <sub>large</sub>                         | 86.6/ -          | 81.8/79.0           |
| RoBERTa <sub>large</sub>                      | 90.2/90.2        | 89.4/86.5           |
| ELECTRA <sub>large</sub> (无准确对应的中文词汇, 直接原词输出) | -                | 90.6/88.0           |
| DeBERTa <sub>large</sub>                      | 91.1/91.1        | 90.7/88.0           |
| DeBERTa <sub>RT D<sub>large</sub></sub>       | 92.0/91.9        | 91.5/89.0           |
| DeBERTa 15 亿参数版                               | 91.7/91.9        | 92.2/89.7           |

表 15: DeBERTa 模型的不同变体在 MNLI 和 SQuAD 2.0 上的比较。